El Taller del Programador

Primer programa de consola en C++: Hola Mundo

Alexis Maier

E.E.S.T. N° 1 "Eduardo Ader" — Vicente López
Laboratorio de Programación (LPR) — 5° Año 3° División — Grupos A-B

Profesor: Prof. Mansilla Muñoz York Elías
Fecha: 01 de octubre de 2026

Introducción

La actividad 2 correspondió al arranque efectivo del Taller del Programador. La consigna pedía configurar el entorno de trabajo, compilar y ejecutar el primer programa de consola propio en C++, comprobando que la cadena de herramientas, formada por el editor, la extensión, el compilador y la salida de ejecución, funcionaba de punta a punta antes de encarar los contenidos de mayor complejidad. Si bien el código resultante es mínimo, la actividad cumple una función diagnóstica: si un problema se presenta en esta instancia, el error está en la configuración y no en la lógica de los ejercicios posteriores.

Desarrollo

Estructura del programa

El archivo *src/Act 2 - main.cpp* declara la biblioteca estándar de entrada y salida mediante *#include <iostream>* y emplea *using namespace std;* para abreviar los comandos de consola. La función *main()* es el punto de entrada obligatorio de todo programa de consola y se declara con tipo de retorno *int*. La salida se construye encadenando operadores <<, que vuelcan cada cadena en la consola, y la finalización se realiza con *return 0;*, convención universal que indica que el programa concluyó correctamente.

Compilación y ejecución

El entorno utilizado fue Visual Studio Code con la extensión Microsoft C/C++ y el compilador GNU. La compilación se ejecutó desde la terminal integrada con la instrucción *g++ "src/Act 2 - main.cpp" -o src/tallermundo.exe*, que genera el ejecutable en el mismo repositorio. La ejecución posterior mediante *.\src\tallermundo.exe* mostró la ventana de consola con el mensaje de bienvenida y el detalle de integrantes. La consigna contemplaba el uso obligatorio de OnlineGDB en los equipos que no dispongan de compilador local, extremo que no fue necesario en esta máquina.

Regla obligatoria de no plagio

La consigna exige modificar la salida de pantalla agregando los nombres y apellidos reales antes de compilar y entregar. La línea de integrantes se personalizó con los tres integrantes del grupo, de modo que el programa identifica de forma inequívoca a sus autores y evita la reutilización directa de la plantilla.

Evidencia fotográfica

Se archivaron en la carpeta *capturas/* las imágenes de la versión del compilador, de la extensión de C/C++ activa en Visual Studio Code y de la ejecución del programa.

Cuestionario Técnico de Control

1. Función de la directiva de inclusión

#include <iostream> es una directiva del preprocesador que inserta el contenido de la biblioteca estándar de entrada y salida en la unidad de traducción, antes de la compilación. Sin ella, el compilador no reconocería *cout* ni *cin*. No es una función: no consume argumentos ni

devuelve valores.

2. Significado de return 0

El código 0 es el valor convencional que devuelve un programa al sistema operativo para indicar que todas las operaciones se completaron sin errores. Cualquier valor distinto de cero se interpreta como fallo. Esta convención permite que el script que lanza el programa decida si debe continuar con los pasos siguientes.

3. Diferencia entre endl y el salto de línea

Ambos producen un salto de línea, pero *endl* además vacía el búfer de salida, lo que fuerza al sistema a escribir los datos en la pantalla de inmediato. Para un programa con pocas líneas de salida esa garantía es innecesaria y el carácter de salto de línea resulta más eficiente; *endl* se reserva para los casos en que se necesita sincronizar la salida con la entrada.

4. Alcance de using namespace std

La directiva importa los nombres del espacio de nombres estándar al ámbito global, permitiendo escribir *cout* en lugar de *std::cout*. Si bien reduce la verborrea, puede generar ambigüedades si existen nombres propios que coincidan con los de la biblioteca estándar, razón por la cual en proyectos reales suele preferirse el prefijo explícito.

5. Autoevaluación

La actividad parecía sencilla, pero permitió comprobar el flujo real de trabajo: editar, guardar, compilar desde la terminal, ejecutar el binario y detectar errores. Ese ciclo se repite en todas las actividades posteriores y resulta mucho más natural cuando se lo practica desde el primer ejercicio.

Conclusión

La actividad cumplió su propósito de dejar el entorno de desarrollo verificado y operativo antes de introducir conceptos de mayor profundidad. Aunque el algoritmo no presenta complejidad lógica, obliga a dominar la cadena completa de herramientas: editor, extensión de lenguaje, compilador, generación del ejecutable y ejecución en consola. Comprender desde el primer día que el código fuente no es el producto final, sino una instrucción dirigida al compilador, resultó ser una base necesaria para las actividades que ejecutan cálculos, estructuras y punteros sobre la memoria.

Referencias

Microsoft Learn. (s. f.). *Introducción a C++*. https://learn.microsoft.com/es-es/cpp/

cppreference.com. (s. f.). *std::cout*. Referencia de C++. https://en.cppreference.com/w/io/cout

Free Software Foundation. (s. f.). *GNU Compiler Collection (GCC)*. https://gcc.gnu.org/

Microsoft. (s. f.). *Visual Studio Code*. Code de Visual Studio. https://code.visualstudio.com/